

Software Requirements Specification

Real Estate Listing Service

Version 1.0

Prepared: September 2026

Document Classification: Internal / Draft

Conforms to IEEE 830 SRS structure

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms and Abbreviations
 - 1.4 References
 - 1.5 Document Overview
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 Assumptions and Dependencies
3. System Features (Functional Requirements)
4. External Interface Requirements
5. Object-Oriented Class Design
 - 5.1 PropertyType Class
 - 5.2 AgentProfile Class
 - 5.3 ClientMatchmaker Class
 - 5.4 Supporting Classes
 - 5.5 Class Relationship Overview
 - 5.6 Class Diagram (UML)
6. UML Diagrams
 - 6.1 Use Case Diagram
 - 6.2 Sequence Diagram
 - 6.3 Collaboration Diagram
 - 6.4 Activity Diagram
 - 6.5 State Chart Diagram
7. Non-Functional Requirements
8. Appendix A: Data Dictionary

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Real Estate Listing Service, a web-based platform that allows property owners and agents to publish listings, allows prospective buyers and renters to search for properties, and automatically matches clients to suitable properties. This document is intended for developers, quality assurance engineers, project managers, and stakeholders involved in the design, implementation, and testing of the system.

1.2 Scope

The Real Estate Listing Service (hereafter "the System") will provide a centralized platform for managing property listings, agent profiles, and client-property matchmaking. The System will support three core object models – PropertyType, AgentProfile, and ClientMatchmaker – along with supporting entities such as Clients, Listings, Inquiries, and Transactions. The System is web- and mobile-accessible and integrates with mapping, payment, and notification services.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
MLS	Multiple Listing Service – an external database of property listings
Listing	A published property record available for sale or rent
Matchmaker	The subsystem/class responsible for pairing clients with relevant properties
Agent	A licensed real estate professional who manages listings and client relationships
Client	An end user seeking to buy, sell, or rent a property

1.4 References

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
Project Charter – Real Estate Listing Service, Internal Document
UML 2.5 Specification, Object Management Group

1.5 Document Overview

Section 2 describes the product context and user classes. Section 3 defines functional requirements. Section 4 covers external interfaces. Section 5 presents the object-oriented class design for Property Types, Agent Profiles, and Client Matchmakers, including attributes, methods, and relationships. Section 6 presents the complete set of UML diagrams modeling the system. Section 7 specifies non-functional requirements, and Section 8 provides a supporting data dictionary.

2. Overall Description

2.1 Product Perspective

The Real Estate Listing Service is a new, self-contained, cloud-hosted platform. It replaces manual/paper-based listing management and disparate spreadsheets currently used by independent agents. It optionally synchronizes with external Multiple Listing Service (MLS) feeds and third-party mapping and payment providers.

2.2 Product Functions

- Create, update, publish, and archive property listings
- Maintain agent profiles, credentials, and performance metrics
- Match clients to properties based on stated preferences and behavior
- Search, filter, and sort listings by location, price, and features
- Send notifications for new matches, price changes, and inquiries
- Track inquiries, showings, offers, and transaction status

2.3 User Classes and Characteristics

User Class	Description
Client (Buyer/Renter)	Searches listings, saves preferences, receives matched recommendations, contacts agents
Agent	Creates and manages listings, responds to inquiries, tracks lead and performance
Administrator	Manages user accounts, moderates listings, configures system settings
Property Owner/Seller	Submits property details to an agent or directly for listing

2.4 Operating Environment

The System operates as a responsive web application accessible via modern browsers (Chrome, Firefox, Safari, Edge) and as native iOS/Android companion apps. The backend runs on a cloud infrastructure (containerized services) with a relational database for transactional data.

2.5 Design and Implementation Constraints

Must comply with Fair Housing and real estate advertising regulations

Must support role-based access control (Client, Agent, Administrator)

Object model must be extensible to add new property categories without schema redesign

2.6 Assumptions and Dependencies

It is assumed that agents possess valid licensing information at registration, that clients provide accurate preference data for matchmaking, and that third-party mapping/geocoding and payment gateway services remain available via API.

3. System Features (Functional Requirements)

FR-1 – Property Listing Management

The System shall allow Agents to create, edit, publish, unpublish, and archive property listings, including photos, pricing, and status (Active, Pending, Sold, Rented, Off-Market).

FR-2 – Agent Profile Management

The System shall allow Agents to maintain a profile including license number, agency affiliation, specialization, service areas, and performance ratings.

FR-3 – Client Preference Capture

The System shall allow Clients to specify search preferences such as budget, location, property type, number of bedrooms/bathrooms, and desired amenities.

FR-4 – Automated Client-Property Matching

The System shall use a ClientMatchmaker component to automatically rank and recommend properties to a Client based on stated preferences and historical interaction data.

FR-5 – Search and Filtering

The System shall allow any user to search listings using keyword, location radius, price range, and property attributes, with results sortable by relevance, price, or date listed.

FR-6 – Notifications

The System shall notify Clients of new matching listings and notify Agents of new inquiries or offers, via email and in-app notification.

FR-7 – Inquiry and Showing Management

The System shall allow Clients to request property showings and Agents to accept, reschedule, or decline such requests.

4. External Interface Requirements

4.1 User Interfaces

The System shall provide a responsive web dashboard for Agents and Administrators, and a consumer-facing search and browsing interface for Clients, following accessible design (WCAG 2.1 AA) guidelines.

4.2 Software Interfaces

- MLS data feed integration for syndicated listings (REST/XML)
- Mapping/geocoding API for address validation and map display
- Payment gateway API for application/deposit fee processing
- Email/SMS gateway for notifications

4.3 Communication Interfaces

All client-server communication shall use HTTPS with TLS 1.2 or higher. Internal services shall communicate over authenticated REST APIs.

5. Object-Oriented Class Design

This section defines the three primary classes requested for the Real Estate Listing Service — PropertyType, AgentProfile, and ClientMatchmaker — along with supporting classes and their relationships. Each class is presented with its attributes, data types, and operations (methods).

5.1 PropertyType Class

Represents a property listing available on the platform. Supports subtypes such as ResidentialProperty, CommercialProperty, and RentalProperty through inheritance.

Attributes

Attribute	Type	Description
propertyId	String	Unique identifier for the property listing
category	Enum	Residential, Commercial, Land, or Rental
address	Address	Full street address and geocoordinates
price	Decimal	Listing price or monthly rent
bedrooms	Integer	Number of bedrooms (if applicable)
bathrooms	Integer	Number of bathrooms (if applicable)
squareFootage	Integer	Total interior area in square feet
status	Enum	Active, Pending, Sold, Rented, or Off-Market
images	List<Image>	Collection of property photographs

listedByAgent	AgentProfile	Reference to the responsible agent
---------------	--------------	------------------------------------

Methods

Method	Returns	Description
createListing()	PropertyType	Instantiates and validates a new listing
updateStatus(status)	void	Changes listing status (e.g., to Sold)
calculatePriceEstimate()	Decimal	Estimates market value using comparable data
addImage(image)	void	Attaches a new photo to the listing
getDetails()	PropertySummary	Returns a formatted summary of the listing
archiveListing()	void	Removes the listing from active search results

5.2 AgentProfile Class

Represents a licensed real estate agent who manages listings and interacts with clients.

Attributes

Attribute	Type	Description
agentId	String	Unique identifier for the agent
fullName	String	Agent's full name
licenseNumber	String	State-issued real estate license number
agencyName	String	Brokerage or agency affiliation
contactInfo	ContactInfo	Phone number, email, and office address
specialization	List<String>	Areas of expertise (e.g., Luxury, Commercial)
serviceAreas	List<String>	Geographic regions served
activeListings	List<PropertyType>	Listings currently managed by the agent
rating	Float	Average client rating (0–5)

Methods

Method	Returns	Description

createListing(details)	PropertyType	Creates a new property listing
respondToInquiry(inquiryId, msg)	void	Sends a response to a client inquiry
scheduleShowing(propertyId, dt)	Showing	Books a property showing appointment
getPerformanceMetrics()	AgentStats	Returns sales/lead conversion statistics

5.3 ClientMatchmaker Class

Encapsulates the logic that pairs a Client with relevant PropertyType listings based on stated preferences, weighted scoring, and historical interaction behavior.

Attributes

Attribute	Type	Description
matchmakerId	String	Unique identifier for the matching session/engine instance
client	Client	Reference to the client being matched
preferences	PreferenceProfile	Budget, location, property type, and amenity preferences
matchedProperties	List<PropertyType>	Ranked list of recommended properties
savedSearches	List<SearchCriteria>	Persisted search filters for recurring alerts
matchThreshold	Float	Minimum relevance score for inclusion in results

Methods

Method	Returns	Description
matchProperties()	List<PropertyType>	Runs the matching algorithm and returns results
updatePreferences(prefs)	void	Updates the client's stated search preferences
rankByRelevance(list)	List<PropertyType>	Sorts candidate properties by match score
notifyNewMatches()	void	Triggers a notification when new matches are found

saveSearch(criteria)	void	Persists a search for future automatic re-matching
calculateMatchScore(property)	Float	Computes a 0–100 relevance score for one property

5.4 Supporting Classes

The following supporting classes are referenced by the three primary classes above and complete the object model.

Class	Purpose
Client	Represents an end user searching for properties; owns a ClientMatchmaker instance
Address	Value object holding street, city, state, zip, and geocoordinates
ContactInfo	Value object holding phone, email, and office address for an agent
Inquiry	Represents a client message or question directed to an agent about a listing
Showing	Represents a scheduled property viewing appointment
PreferenceProfile	Stores a client's budget, location, property type, and amenity criteria
SearchCriteria	A saved, reusable set of search filters

5.5 Class Relationship Overview

Relationship	Cardinality	Description
AgentProfile → PropertyType	1 to many	An agent manages zero or more property listings
ClientMatchmaker → Client	1 to 1	Each matchmaker instance is bound to one client
ClientMatchmaker → PropertyType	1 to many	A matchmaker produces a ranked list of matched properties
Client → Inquiry	1 to many	A client may submit multiple inquiries
PropertyType → Inquiry	1 to many	A listing may receive multiple inquiries
AgentProfile → Showing	1 to many	An agent may host multiple scheduled showings

Note: PropertyType is designed as a base class, extensible via subclasses (e.g., ResidentialProperty, CommercialProperty, LandProperty, RentalProperty) that inherit its core attributes and override category-specific behavior.

5.6 Class Diagram (UML)

The diagram below presents the UML class model of the platform's core entities – User (with Client, Agent, and Admin specializations), Property, Listing, Booking, Payment, and Review – illustrating the structural relationships underpinning the PropertyType, AgentProfile, and ClientMatchmaker design described above

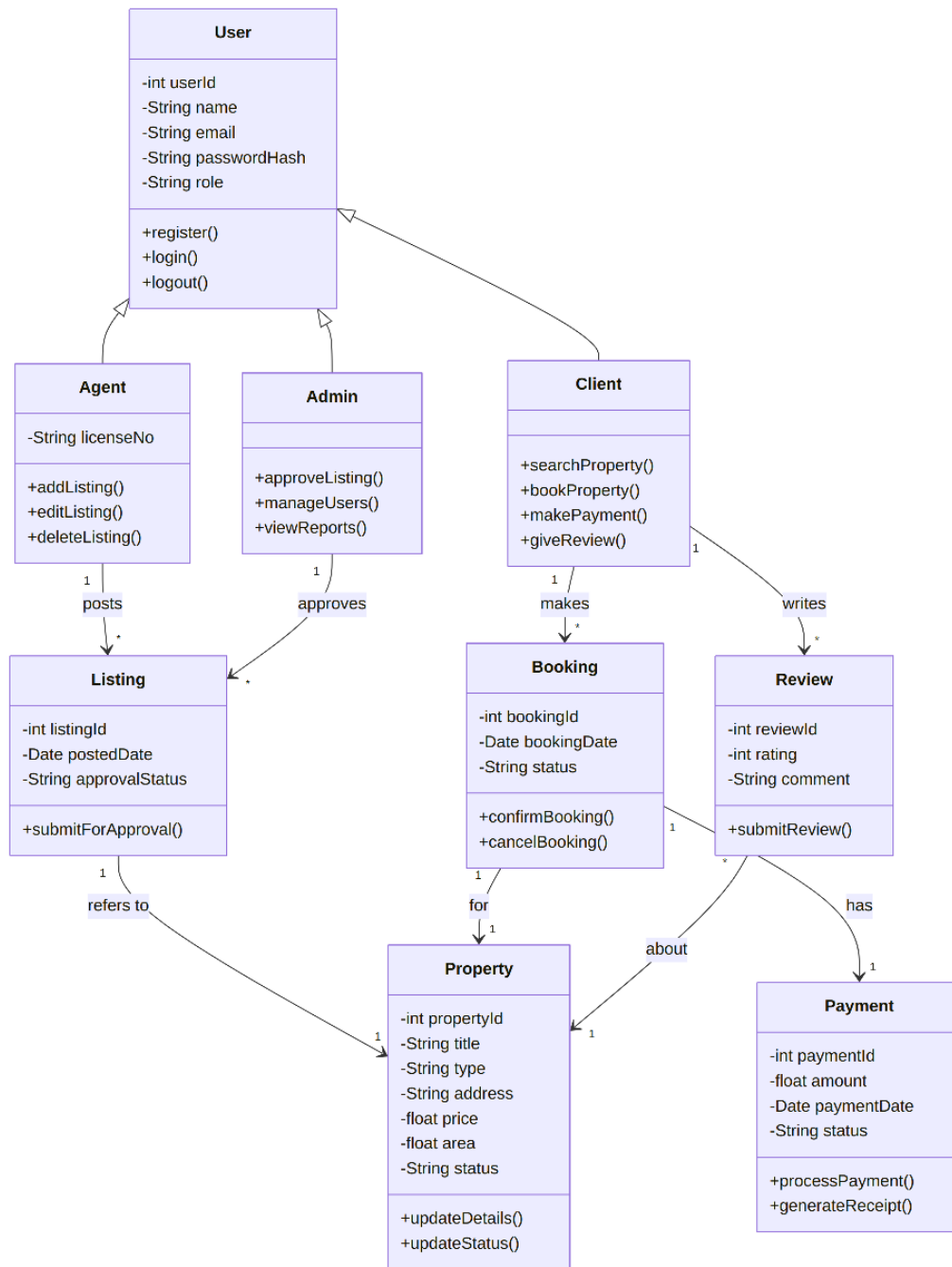


Figure 5.1: UML Class Diagram – Real Estate Listing Service

6. UML Diagrams

This section presents the complete set of UML diagrams modeling the structure and behavior of the Real Estate Listing Service: Use Case, Sequence, Collaboration, Activity, and State Chart diagrams.

6.1 Use Case Diagram

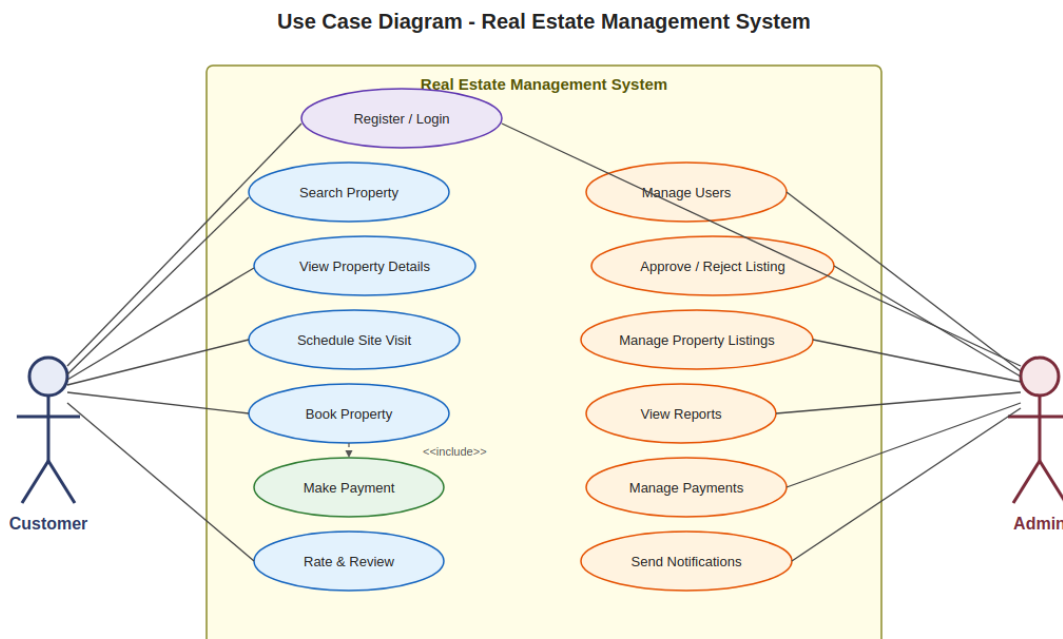


Figure 6.1 Use Case Diagram – Customer and Admin Interactions

Customers search, view, schedule visits, book, pay, and review properties. Administrators manage users, approve or reject listings, manage property listings, view reports, manage payments, and send notifications.

6.2 Sequence Diagram – Property Booking

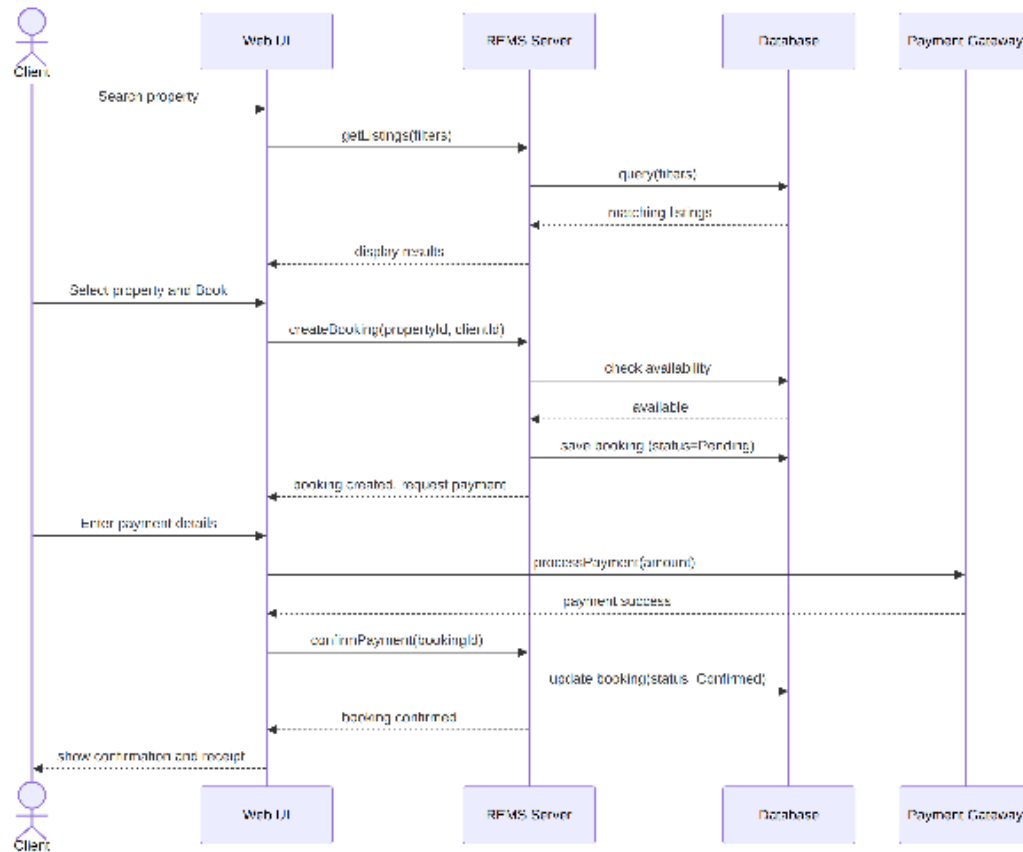


Figure 6.2: Sequence Diagram –Property Search and Booking Flow

Depicts the time-ordered interaction between the Client, Web UI, REMS Server, Database, and Payment Gateway during a property search-and-booking flow, corresponding to the `matchProperties()` and related operations of the `ClientMatchmaker` class.

6.3 Collaboration Diagram –Property Booking

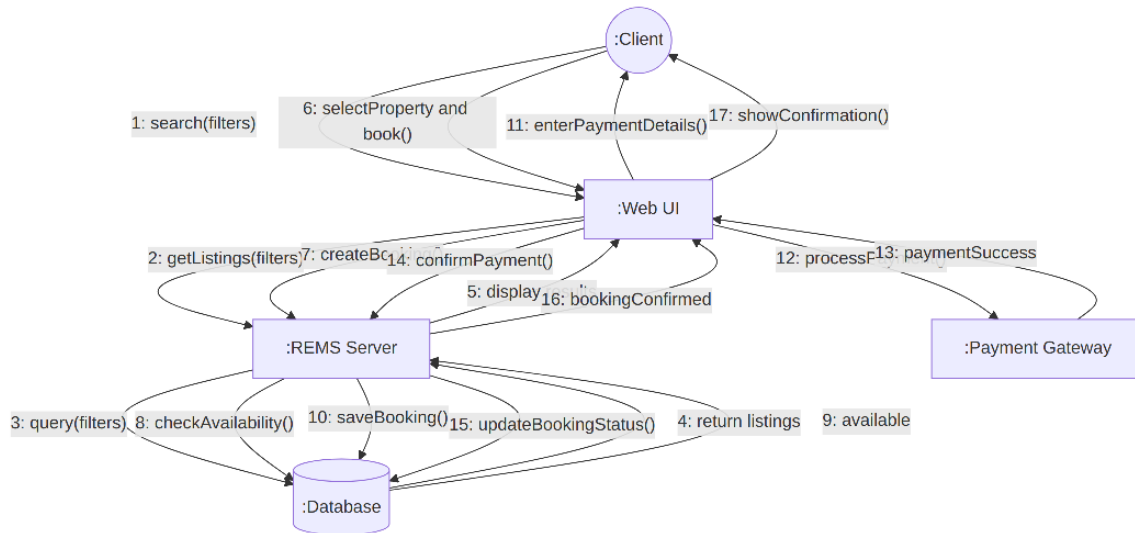


Figure 6.3: Collaboration Diagram – Property Booking

Represents the same booking interaction as an object-relationship structure, with numbered messages showing the order of communication between collaborating objects.

6.4 Activity Diagram – Property Booking Process

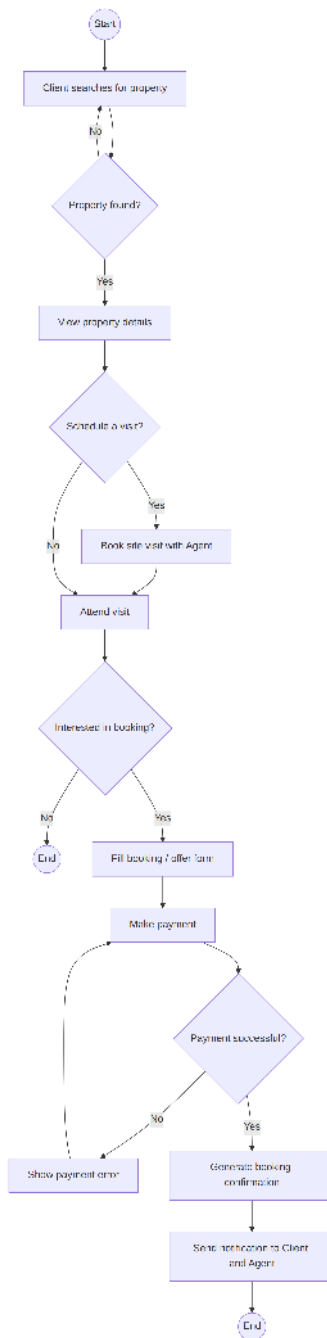


Figure 6.4: Activity Diagram –Booking Workflow

Models the workflow and decision points a Client follows: searching, viewing details, scheduling a visit, booking, paying, and receiving confirmation.

6.5 State Chart Diagram –Property Listing Lifecycle

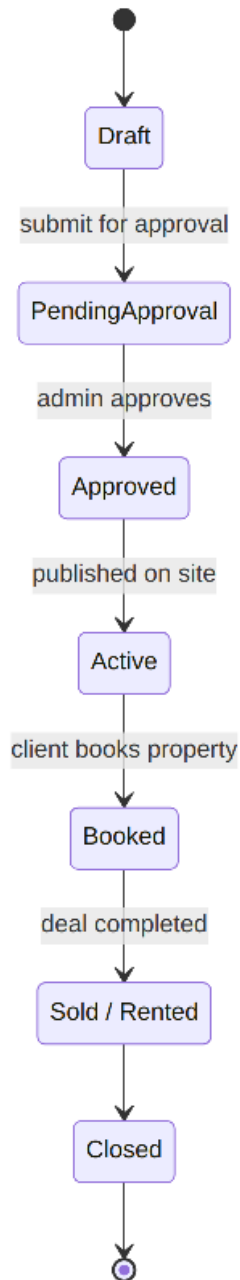


Figure 6.5: State Chart Diagram – Property Listing Lifecycle

Tracks a property listing through its lifecycle states – Draft, Pending Approval, Approved, Active, Booked, Sold/Rented, and Closed – corresponding to the status attribute of the PropertyType class.

7. Non-Functional Requirements

Performance

The System shall return search results within 2 seconds for 95% of queries under normal load.

Scalability

The System shall support at least 100,000 active listings and 50,000 concurrent users.

Security

The System shall encrypt data in transit and at rest, and enforce role-based access control.

Availability

The System shall maintain 99.5% uptime, excluding scheduled maintenance windows.

Usability

New users shall be able to complete a property search without training or assistance.

Compliance

The System shall comply with applicable Fair Housing and data privacy regulations.

Maintainability

The class model shall support adding new property categories with minimal code changes.

8. Appendix A: Data Dictionary

Field	Type	Notes
propertyId / agentId / dierId	String (UUID)	Globally unique identifiers
price	Decimal (2 dp)	Stored in USD; currency configurable per locale
status	Enum	Active Pending Sold Rented Off-Market
rating	Float	Range 0.0–5.0
matchThreshold	Float	Range 0.0–100.0

End of Document